



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ТЕОРИЯ ИГР И ИССЛЕДОВАНИЕ ОПЕРАЦИЙ

Лабораторная работа № 8

на тему:

«Байесовские игры. Статическая и динамическая постановки»

Вариант 13

Студент:

ИУ8-105

(Группа)

Б. Е. Шекунов

(И. О. Фамилия)

Преподаватель:

Н. С. Коннова

(И. О. Фамилия)

Москва, 2026 г.

Цель работы:

Цель работы – изучить постановку статических и динамических байесовских игр с неполной информацией, найти равновесия Байеса-Нэша для статической игры и совершенное байесовское равновесие для динамической игры.

Постановка задачи:

Рассматривается байесовская игра двух игроков с неполной информацией. Первый игрок не знает точный тип второго игрока, но знает вероятности возможных типов. Тип второго игрока влияет на платежную матрицу игры. В общем виде заданы:

$I = \{1, 2\}$ – множество игроков;

$A_1 = \{a_1, a_2\}, A_2 = \{b_1, b_2\}$ – множества действий игроков;

$T_2 = \{\theta_1, \theta_2\}$ – множество типов второго игрока;

$P(\theta_1) = p, P(\theta_2) = 1 - p$ – вероятности типов второго игрока.

Необходимо рассмотреть две постановки игры:

Статическая байесовская игра. Оба игрока выбирают действия одновременно. Первый игрок выбирает одну из стратегий a_1, a_2 , а второй игрок выбирает действие в зависимости от своего типа. Стратегия второго игрока имеет вид:

$$s_2: T_2 \rightarrow A_2.$$

Требуется найти чистые равновесия Байеса-Нэша и ожидаемые выигрыши игроков.

Динамическая байесовская игра. Сначала Природа выбирает тип второго игрока, затем первый игрок выбирает действие, после чего второй игрок, зная свой тип и наблюдая действие первого игрока, выбирает ответ. Стратегия второго игрока имеет вид:

$$s_2: T_2 \times A_1 \rightarrow A_2.$$

Требуется найти совершенное байесовское равновесие и ожидаемые выигрыши игроков.

Исходные данные:

Пусть:

$$\theta_1 = v13, \theta_2 = v14.$$

Вероятности типов:

$$P(v13) = p, P(v14) = 1 - p.$$

В основном расчёте принято:

$$p = 0.5.$$

Матрица выплат для типа $v13$:

$$G_{v13} = \begin{pmatrix} (4,1) & (6,2) \\ (11,7) & (0,5) \end{pmatrix}.$$

Матрица выплат для типа $v14$:

$$G_{v14} = \begin{pmatrix} (9,8) & (7,4) \\ (2,1) & (10,3) \end{pmatrix}.$$

В каждой паре первое число – выигрыш первого игрока, второе число – выигрыш второго игрока. Строки соответствуют действиям первого игрока a_1, a_2 , столбцы – действиям второго игрока b_1, b_2 .

Ход работы:

В статической игре первый игрок выбирает одно действие:

$$s_1 \in \{a_1, a_2\}.$$

Второй игрок знает свой тип, поэтому его стратегия задаётся как действие для каждого типа:

$$s_2 = (v13 \rightarrow b_i, v14 \rightarrow b_j).$$

Рассмотрим возможные лучшие ответы второго игрока.

Если первый игрок выбирает a_1 , то для типа $v13$:

$$u_2(b_1) = 1, u_2(b_2) = 2.$$

Следовательно, типу $v13$ выгодно выбрать b_2 .

Для типа $v14$:

$$u_2(b_1) = 8, u_2(b_2) = 4.$$

Следовательно, типу $v14$ выгодно выбрать b_1 .

Получаем кандидат на равновесие:

$$s_1 = a_1, s_2 = (v13 \rightarrow b_2, v14 \rightarrow b_1).$$

Ожидаемый выигрыш первого игрока при $p = 0.5$:

$$U_1(a_1) = 0.5 \cdot 6 + 0.5 \cdot 9 = 7.5.$$

При отклонении первого игрока к a_2 :

$$U_1(a_2) = 0.5 \cdot 0 + 0.5 \cdot 2 = 1.$$

Так как $7.5 > 1$ первому игроку невыгодно отклоняться. Поэтому найден первый профиль равновесия:

$$(a_1; v13 \rightarrow b_2, v14 \rightarrow b_1).$$

Ожидаемые выигрыши:

$$E[u] = (7.5, 5).$$

Теперь рассмотрим случай, когда первый игрок выбирает a_2 .

Для типа $v13$:

$$u_2(b_1) = 7, u_2(b_2) = 5.$$

Следовательно, типу $v13$ выгодно выбрать b_1 .

Для типа $v14$:

$$u_2(b_1) = 1, u_2(b_2) = 3.$$

Следовательно, типу $v14$ выгодно выбрать b_2 .

Получаем второй кандидат:

$$s_1 = a_2, s_2 = (v13 \rightarrow b_1, v14 \rightarrow b_2).$$

Ожидаемый выигрыш первого игрока:

$$U_1(a_2) = 0.5 \cdot 11 + 0.5 \cdot 10 = 10.5.$$

При отклонении к a_1 :

$$U_1(a_1) = 0.5 \cdot 4 + 0.5 \cdot 7 = 5.5.$$

Так как $10.5 > 5.5$ первому игроку невыгодно отклоняться. Поэтому найден второй профиль равновесия:

$$(a_2; v13 \rightarrow b_1, v14 \rightarrow b_2).$$

Ожидаемые выигрыши:

$$E[u] = (10.5, 5).$$

Таким образом, в статической байесовской игре при $p = 0.5$ найдено два чистых равновесия Байеса-Нэша:

$$(a_1; v13 \rightarrow b_2, v14 \rightarrow b_1),$$

$$(a_2; v13 \rightarrow b_1, v14 \rightarrow b_2).$$

Для первого равновесия:

$$s_1 = a_1, s_2 = (v13 \rightarrow b_2, v14 \rightarrow b_1).$$

Ожидаемый выигрыш первого игрока:

$$U_1(a_1) = 6p + 9(1 - p) = 9 - 3p.$$

При отклонении к a_2 :

$$U_1(a_2) = 0p + 2(1 - p) = 2 - 2p.$$

Условие равновесия:

$$9 - 3p \geq 2 - 2p.$$

$$p \leq 7.$$

Так как $p \in [0,1]$, условие выполняется для всех допустимых значений p .

Для второго равновесия:

$$s_1 = a_2, s_2 = (v13 \rightarrow b_1, v14 \rightarrow b_2).$$

Ожидаемый выигрыш первого игрока:

$$U_1(a_2) = 11p + 10(1 - p) = 10 + p.$$

При отклонении к a_1 :

$$U_1(a_1) = 4p + 7(1 - p) = 7 - 3p.$$

Условие равновесия:

$$10 + p \geq 7 - 3p.$$

$$p \geq -\frac{3}{4}.$$

Это условие также выполняется для всех $p \in [0,1]$.

Статическая игра имеет два чистых равновесия Байеса-Нэша.

В динамической постановке порядок ходов следующий:

1. Природа выбирает тип второго игрока: $v13$ или $v14$.
2. Первый игрок выбирает действие a_1 или a_2 , не зная тип второго игрока.
3. Второй игрок знает свой тип, наблюдает действие первого игрока и выбирает b_1 или b_2 .

Сначала найдём лучшие ответы второго игрока в каждой возможной ситуации.

Для типа $v13$:

если первый игрок выбрал a_1 :

$$u_2(b_1) = 1, u_2(b_2) = 2,$$

поэтому:

$$s_2(v13, a_1) = b_2.$$

Если первый игрок выбрал a_2 :

$$u_2(b_1) = 7, u_2(b_2) = 5,$$

поэтому:

$$s_2(v13, a_2) = b_1.$$

Для типа $v14$:

если первый игрок выбрал a_1 :

$$u_2(b_1) = 8, u_2(b_2) = 4,$$

поэтому:

$$s_2(v14, a_1) = b_1.$$

Если первый игрок выбрал a_2 :

$$u_2(b_1) = 1, u_2(b_2) = 3,$$

поэтому:

$$s_2(v14, a_2) = b_2.$$

Итак, рациональная стратегия второго игрока:

$$s_2(v13, a_1) = b_2,$$

$$s_2(v13, a_2) = b_1,$$

$$s_2(v14, a_1) = b_1,$$

$$s_2(v14, a_2) = b_2.$$

Теперь первый игрок сравнивает ожидаемые выигрыши от действий a_1 и a_2 .

Если первый игрок выбирает a_1 , то:

$$U_1(a_1) = 6p + 9(1 - p) = 9 - 3p.$$

Если первый игрок выбирает a_2 , то:

$$U_1(a_2) = 11p + 10(1 - p) = 10 + p.$$

Первому игроку выгоднее выбрать a_2 , если:

$$10 + p \geq 9 - 3p$$

$$p \geq -\frac{1}{4}$$

Так как в задаче $p \in [0,1]$, это условие выполняется для всех допустимых значений p . Следовательно, в данной динамической игре первый игрок при любом $p \in [0,1]$ выбирает a_2 . То есть порогового значения вероятности внутри допустимого интервала нет.

Например, при $p = 0.5$:

$$U_1(a_1) = 7.5,$$

$$U_1(a_2) = 10.5.$$

Так как $10.5 > 7.5$, первому игроку выгоднее выбрать a_2 .

Ожидаемые выигрыши при $p = 0.5$:

$$E[u] = (10.5, 5).$$

Совершенное байесовское равновесие динамической игры имеет вид:

$$s_1 = a_2,$$

$$s_2(v13, a_1) = b_2,$$

$$s_2(v13, a_2) = b_1,$$

$$s_2(v14, a_1) = b_1,$$

$$s_2(v14, a_2) = b_2.$$

Вывод:

В работе была рассмотрена байесовская игра с неполной информацией.

В статической постановке первый игрок выбирает действие, не зная тип второго игрока, а второй игрок выбирает действие в зависимости от своего типа. При $p = 0.5$ найдено два чистых равновесия Байеса-Нэша:

$$(a_1; v13 \rightarrow b_2, v14 \rightarrow b_1),$$

$$(a_2; v13 \rightarrow b_1, v14 \rightarrow b_2).$$

Ожидаемые выигрыши в этих равновесиях равны:

$$E[u] = (7.5, 5), E[u] = (10.5, 5).$$

Во втором равновесии первый игрок получает больший ожидаемый выигрыш.

В динамической постановке игра была рассмотрена как последовательная: сначала ходит первый игрок, затем второй игрок, зная свой тип и наблюдая действие первого игрока. Решение было найдено методом обратного рассуждения: сначала определены лучшие ответы второго игрока, затем выбран оптимальный ход первого игрока.

В динамической игре найдено одно совершенное байесовское равновесие:

$$Player1 \rightarrow a_2.$$

При этом второй игрок действует следующим образом:

$$s_2(v13, a_1) = b_2, s_2(v13, a_2) = b_1,$$

$$s_2(v14, a_1) = b_1, s_2(v14, a_2) = b_2.$$

Ожидаемые выигрыши равны:

$$E[u] = (10.5, 5).$$

Таким образом, переход от статической к динамической постановке изменяет множество равновесий: в статической игре существует два равновесия, а в динамической остаётся одно равновесие, выбранное с учётом последовательной рациональности второго игрока.

Приложение (исходный код)

```
from __future__ import annotations

from dataclasses import dataclass
from itertools import product
from typing import Dict, Hashable, List, Mapping, Tuple

Action = Hashable
TypeName = Hashable
Player = str
TypeProfile = Tuple[TypeName, ...]
ActionProfile = Tuple[Action, ...]
Payoff = Tuple[float, ...]

def fmt_number(x: float, digits: int = 6) -> str:
    if abs(x - round(x)) < 10 ** (-(digits - 1)):
        return str(int(round(x)))
    return f"{x:.{digits}f}".rstrip("0").rstrip(".")

def fmt_payoff(payoff: Tuple[float, ...]) -> str:
    return "(" + ", ".join(fmt_number(v) for v in payoff) + ")"

@dataclass(frozen=True)
class BayesianGame:
    players: Tuple[Player, ...]
    types: Tuple[Tuple[TypeName, ...], ...]
    actions: Tuple[Tuple[Action, ...], ...]
    prior: Mapping[TypeProfile, float]
    payoff: Mapping[TypeProfile, ActionProfile, Payoff]

    def __post_init__(self) -> None:
        if len(self.players) != len(self.types) or len(self.players) != len(self.actions):
            raise ValueError("players, types и actions должны иметь одинаковую длину")

        n = len(self.players)
        total = sum(self.prior.values())
        if abs(total - 1.0) > 1e-9:
            raise ValueError(f"Сумма вероятностей типов должна быть 1, сейчас {total}")

        all_type_profiles = set(product(*self.types))
        if set(self.prior.keys()) != all_type_profiles:
            missing = all_type_profiles - set(self.prior.keys())
            extra = set(self.prior.keys()) - all_type_profiles
            raise ValueError(
                "prior должен быть задан для всех профилей типов; "
                f"missing={missing}, extra={extra}"
            )

        for type_profile in all_type_profiles:
            for action_profile in product(*self.actions):
                key = (type_profile, action_profile)
                if key not in self.payoff:
                    raise ValueError(
                        f"Нет платежей для type_profile={type_profile}, "
                        f"action_profile={action_profile}"
                    )
                if len(self.payoff[key]) != n:
                    raise ValueError(f"Платеж должен содержать {n} чисел")

    def pure_strategies_for_player(self, player_index: int) -> List[Tuple[Action, ...]]:
        return list(product(self.actions[player_index], repeat=len(self.types[player_index])))

    def all_strategy_profiles(self) -> List[Tuple[Tuple[Action, ...], ...]]:
        strategies = [self.pure_strategies_for_player(i) for i in range(len(self.players))]
        return list(product(*strategies))

    def action_by_strategy(self, player_index: int, strategy: Tuple[Action, ...], type_i:
        TypeName) -> Action:
        type_position = self.types[player_index].index(type_i)
        return strategy[type_position]

    def action_profile_from_strategies(
        self,
        strategy_profile: Tuple[Tuple[Action, ...], ...],

```

```

        type_profile: TypeProfile,
    ) -> ActionProfile:
        return tuple(
            self.action_by_strategy(i, strategy_profile[i], type_profile[i])
            for i in range(len(self.players))
        )

    def conditional_type_profiles(self, player_index: int, own_type: TypeName) ->
List[Tuple[TypeProfile, float]]:
        denom = sum(prob for tp, prob in self.prior.items() if tp[player_index] == own_type)
        if denom <= 0:
            raise ValueError(f"Тип {own_type!r} игрока {self.players[player_index]} имеет нулевую
вероятность")
        return [(tp, prob / denom) for tp, prob in self.prior.items() if tp[player_index] ==
own_type]

    def expected_payoff_for_type_action(
        self,
        player_index: int,
        own_type: TypeName,
        action: Action,
        opponents_strategy_profile: Tuple[Tuple[Action, ...], ...],
    ) -> float:
        n = len(self.players)
        total = 0.0

        for type_profile, cond_prob in self.conditional_type_profiles(player_index, own_type):
            action_profile: list[Action] = []
            opponent_pos = 0

            for j in range(n):
                if j == player_index:
                    action_profile.append(action)
                else:
                    strategy_j = opponents_strategy_profile[opponent_pos]
                    action_profile.append(self.action_by_strategy(j, strategy_j,
type profile[j]))
                    opponent_pos += 1

            total += cond_prob * self.payoff[(type_profile, tuple(action_profile))][player_index]

        return total

    def best_responses_for_type(
        self,
        player_index: int,
        own_type: TypeName,
        opponents_strategy_profile: Tuple[Tuple[Action, ...], ...],
        tol: float = 1e-9,
    ) -> Tuple[float, List[Action], Dict[Action, float]]:
        utilities = {
            action: self.expected_payoff_for_type_action(player_index, own_type, action,
opponents_strategy_profile)
            for action in self.actions[player_index]
        }
        best_value = max(utilities.values())
        best_actions = [action for action, value in utilities.items() if abs(value - best_value)
<= tol]
        return best_value, best_actions, utilities

    def is_bayes_nash_equilibrium(
        self,
        strategy_profile: Tuple[Tuple[Action, ...], ...],
        tol: float = 1e-9,
    ) -> bool:
        n = len(self.players)
        for i in range(n):
            opponents = tuple(strategy_profile[j] for j in range(n) if j != i)
            for own_type in self.types[i]:
                chosen = self.action_by_strategy(i, strategy_profile[i], own_type)
                _, best_actions, _ = self.best_responses_for_type(i, own_type, opponents,
tol=tol)
                if chosen not in best_actions:
                    return False
        return True

    def bayes_nash_equilibria(self, tol: float = 1e-9) -> List[Tuple[Tuple[Action, ...], ...]]:
        return [
            strategy_profile

```

```

        for strategy_profile in self.all_strategy_profiles()
        if self.is_bayes_nash_equilibrium(strategy_profile, tol=tol)
    ]

    def ex_ante_expected_payoff(self, strategy_profile: Tuple[Tuple[Action, ...], ...]) ->
    Tuple[float, ...]:
        n = len(self.players)
        total = [0.0] * n

        for type_profile, prob in self.prior.items():
            action_profile = self.action_profile_from_strategies(strategy_profile, type_profile)
            payoff = self.payoff[(type_profile, action_profile)]
            for i in range(n):
                total[i] += prob * payoff[i]

        return tuple(total)

    def describe_strategy_profile(self, strategy_profile: Tuple[Tuple[Action, ...], ...]) -> str:
        parts: list[str] = []
        for i, player in enumerate(self.players):
            mapping = ", ".join(f"{type_i}->{action}" for type_i, action in zip(self.types[i],
            strategy_profile[i]))
            parts.append(f"{player}: ({mapping})")
        return "; ".join(parts)

    def print_equilibrium_analysis(self, title: str) -> None:
        print("\n" + "=" * 92)
        print(title)
        print("=" * 92)

        equilibria = self.bayes_nash_equilibria()

        print(f"Игроки: {self.players}")
        print("Типы:")
        for player, types_i in zip(self.players, self.types):
            print(f"    {player}: {types_i}")
        print("Действия:")
        for player, actions_i in zip(self.players, self.actions):
            print(f"    {player}: {actions_i}")
        print("Априорное распределение типов:")
        for type_profile, prob in sorted(self.prior.items(), key=lambda item: str(item[0])):
            print(f"    P{type_profile} = {fmt_number(prob)}")

        print(f"\nНайдено чистых равновесий Байеса – Нэша: {len(equilibria)}")
        for idx, equilibrium in enumerate(equilibria, start=1):
            print(f"    {idx}) {self.describe_strategy_profile(equilibrium)}")
            print(f"        Ex ante E[u] = {fmt_payoff(self.ex_ante_expected_payoff(equilibrium))}")
            print(f"        Проверка по типам:")

            for i, player in enumerate(self.players):
                opponents = tuple(equilibrium[j] for j in range(len(self.players)) if j != i)
                for own_type in self.types[i]:
                    chosen = self.action_by_strategy(i, equilibrium[i], own_type)
                    _, best_actions, utilities = self.best_responses_for_type(i, own_type,
                    opponents)
                    utilities_s = ", ".join(f"{action}: {fmt_number(value)}" for action, value in
                    utilities.items())
                    print(
                        f"        {player}, тип {own_type}: выбрано {chosen}; "
                        f"        полезности {{{utilities_s}}}; лучшие ответы {best_actions}"
                    )

    @dataclass(frozen=True)
    class DynamicTwoPlayerBayesianGame:
        player1: Player
        player2: Player
        player1_actions: Tuple[Action, ...]
        player2_actions: Tuple[Action, ...]
        player2_types: Tuple[TypeName, ...]
        prior: Mapping[TypeName, float]
        payoff: Mapping[Tuple[TypeName, Action, Action], Payoff]

        def __post_init__(self) -> None:
            total = sum(self.prior.values())
            if abs(total - 1.0) > 1e-9:
                raise ValueError(f"Сумма вероятностей типов должна быть 1, сейчас {total}")

            if set(self.prior.keys()) != set(self.player2_types):

```

```

        raise ValueError("prior должен быть задан для всех типов игрока 2")

    for type_2 in self.player2_types:
        for a in self.player1_actions:
            for b in self.player2_actions:
                key = (type_2, a, b)
                if key not in self.payoff:
                    raise ValueError(f"Нет платежей для type={type_2}, a={a}, b={b}")
                if len(self.payoff[key]) != 2:
                    raise ValueError("Платеж должен содержать два числа")

    def player2_utilities(self, type_2: TypeName, action1: Action) -> Dict[Action, float]:
        return {
            action2: self.payoff[(type_2, action1, action2)][1]
            for action2 in self.player2_actions
        }

    def player2_best_responses(self, type_2: TypeName, action1: Action, tol: float = 1e-9) -> List[Action]:
        utilities = self.player2_utilities(type_2, action1)
        best_value = max(utilities.values())
        return [action for action, value in utilities.items() if abs(value - best_value) <= tol]

    def all_sequentially_rational_player2_strategies(self) -> List[Dict[Tuple[TypeName, Action], Action]]:
        infosets: list[tuple[TypeName, Action]] = [
            (type_2, action1)
            for type_2 in self.player2_types
            for action1 in self.player1_actions
        ]
        best_actions_by_infoset = [self.player2_best_responses(type_2, action1) for type_2,
        action1 in infosets]

        strategies: list[dict[tuple[TypeName, Action], Action]] = []
        for chosen_actions in product(*best_actions_by_infoset):
            strategies.append(dict(zip(infosets, chosen_actions)))
        return strategies

    def player1_expected_payoff(self, action1: Action,
                                player2_strategy: Mapping[Tuple[TypeName, Action], Action]) -> float:
        total = 0.0
        for type_2, prob in self.prior.items():
            action2 = player2_strategy[(type_2, action1)]
            total += prob * self.payoff[(type_2, action1, action2)][0]
        return total

    def expected_payoff(self, action1: Action, player2_strategy: Mapping[Tuple[TypeName, Action], Action]) -> Payoff:
        total_1 = 0.0
        total_2 = 0.0
        for type_2, prob in self.prior.items():
            action2 = player2_strategy[(type_2, action1)]
            p1, p2 = self.payoff[(type_2, action1, action2)]
            total_1 += prob * p1
            total_2 += prob * p2
        return total_1, total_2

    def pure_perfect_bayesian_equilibria(self, tol: float = 1e-9) -> List[Tuple[Action, Dict[Tuple[TypeName, Action], Action]]]:
        equilibria: list[tuple[Action, dict[tuple[TypeName, Action], Action]]] = []

        for player2_strategy in self.all_sequentially_rational_player2_strategies():
            utilities_1 = {
                action1: self.player1_expected_payoff(action1, player2_strategy)
                for action1 in self.player1_actions
            }
            best_value = max(utilities_1.values())
            best_actions_1 = [
                action for action, value in utilities_1.items()
                if abs(value - best_value) <= tol
            ]
            for action1 in best_actions_1:
                equilibria.append((action1, player2_strategy))

        return equilibria

    def describe_player2_strategy(self, strategy: Mapping[Tuple[TypeName, Action], Action]) -> str:

```

```

        chunks = []
        for type_2 in self.player2_types:
            rules = ", ".join(
                f"если {self.player1}->{action1}, то {action2}"
                for action1 in self.player1_actions
                for (t, a), action2 in strategy.items()
                if t == type_2 and a == action1
            )
            chunks.append(f"{self.player2}, тип {type_2}: {rules}")
        return "; ".join(chunks)

def print_backward_induction_solution(self, title: str) -> None:
    print("\n" + "=" * 92)
    print(title)
    print("=" * 92)
    print("Порядок ходов: Природа выбирает тип игрока 2 -> игрок 1 выбирает действие -> игрок 2 выбирает ответ")
    print(f"Игрок 1: {self.player1}, действия: {self.player1_actions}")
    print(f"Игрок 2: {self.player2}, действия: {self.player2_actions}, типы: {self.player2_types}")
    print("Априорные вероятности типов игрока 2:")
    for type_2, prob in self.prior.items():
        print(f"    P({type_2}) = {fmt_number(prob)}")

    print("\nШаг 1. Лучшие ответы игрока 2 в каждой возможной вершине:")
    for type_2 in self.player2_types:
        for action1 in self.player1_actions:
            utilities = self.player2_utilities(type_2, action1)
            best = self.player2_best_responses(type_2, action1)
            utilities_s = ", ".join(f"{a2}: {fmt_number(v)}" for a2, v in utilities.items())
            print(f"    тип {type_2}, после {self.player1}->{action1}: {{{utilities_s}}}; лучшие ответы {best}")

    p2_strategies = self.all_sequentially_rational_player2_strategies()
    print("\nШаг 2. Стратегии игрока 2, рациональные во всех его информационных множествах:")
    for idx, strategy in enumerate(p2_strategies, start=1):
        print(f"    {idx}) {self.describe_player2_strategy(strategy)}")

    print("\nШаг 3. Выбор игрока 1 с учетом ожидаемого выигрыша:")
    equilibria = self.pure_perfect_bayesian_equilibria()
    for idx, strategy in enumerate(p2_strategies, start=1):
        utilities_1 = {
            action1: self.player1_expected_payoff(action1, strategy)
            for action1 in self.player1_actions
        }
        utilities_s = ", ".join(f"{a1}: {fmt_number(v)}" for a1, v in utilities_1.items())
        print(f"    Для стратегии игрока 2 №{idx}: выигрыши игрока 1 {{{utilities_s}}}")

    print(f"\nНайдено чистых совершенных байесовских равновесий: {len(equilibria)}")
    for idx, (action1, strategy) in enumerate(equilibria, start=1):
        print(f"    {idx}) {self.player1}->{action1}; {self.describe_player2_strategy(strategy)}")
        print(f"        Ex ante E[u] = {fmt_payoff(self.expected_payoff(action1, strategy))}")

VARIANT_13_MATRIX: Dict[Tuple[Action, Action], Payoff] = {
    ("a1", "b1"): (4, 1),
    ("a1", "b2"): (6, 2),
    ("a2", "b1"): (11, 7),
    ("a2", "b2"): (0, 5),
}

VARIANT_14_MATRIX: Dict[Tuple[Action, Action], Payoff] = {
    ("a1", "b1"): (9, 8),
    ("a1", "b2"): (7, 4),
    ("a2", "b1"): (2, 1),
    ("a2", "b2"): (10, 3),
}

def print_matrix(title: str, matrix: Mapping[Tuple[Action, Action], Payoff]) -> None:
    print(title)
    print("      b1      b2")
    print(f"a1      {fmt_payoff(matrix[('a1', 'b1')]):>8} {fmt_payoff(matrix[('a1', 'b2')]):>8}")
    print(f"a2      {fmt_payoff(matrix[('a2', 'b1')]):>8} {fmt_payoff(matrix[('a2', 'b2')]):>8}")

def build_static_methodical_variants_13_14_game(p_variant13: float = 0.5) -> BayesianGame:
    if not (0.0 < p_variant13 < 1.0):

```

```

        raise ValueError(
            "Для этого примера используйте  $0 < p_{\text{variant13}} < 1$ , чтобы оба типа имели положительную вероятность")

    players = ("Player1", "Player2")
    types = (("single",), ("v13", "v14"))
    actions = (("a1", "a2"), ("b1", "b2"))
    prior = {
        ("single", "v13"): float(p_variant13),
        ("single", "v14"): float(1 - p_variant13),
    }

    payoff: dict[tuple[TypeProfile, ActionProfile], Payoff] = {}
    for action_profile, p in VARIANT_13_MATRIX.items():
        payoff[("single", "v13"), action_profile] = p
    for action_profile, p in VARIANT_14_MATRIX.items():
        payoff[("single", "v14"), action_profile] = p

    return BayesianGame(players, types, actions, prior, payoff)

def build_dynamic_methodical_variants_13_14_game(p_variant13: float = 0.5) ->
DynamicTwoPlayerBayesianGame:
    if not (0.0 <= p_variant13 <= 1.0):
        raise ValueError("p variant13 должна лежать в [0, 1]")

    payoff: dict[tuple[TypeName, Action, Action], Payoff] = {}
    for (a, b), p in VARIANT_13_MATRIX.items():
        payoff[("v13", a, b)] = p
    for (a, b), p in VARIANT_14_MATRIX.items():
        payoff[("v14", a, b)] = p

    return DynamicTwoPlayerBayesianGame(
        player1="Player1",
        player2="Player2",
        player1_actions=("a1", "a2"),
        player2_actions=("b1", "b2"),
        player2_types=("v13", "v14"),
        prior={"v13": float(p_variant13), "v14": float(1 - p_variant13)},
        payoff=payoff,
    )

def print_static_threshold_for_methodical_game() -> None:
    print("\n" + "-" * 92)
    print("Аналитический разбор статической игры из вариантов 13 и 14")
    print("Обозначим  $p = P(v13)$ ,  $1-p = P(v14)$ .")
    print("Матрица типа v13:")
    print_matrix("", VARIANT_13_MATRIX)
    print("Матрица типа v14:")
    print_matrix("", VARIANT_14_MATRIX)

    print("\nЕсли Player1 выбирает a1:")
    print(" для типа v13 игроку 2 выгоднее b2, так как  $2 > 1$ ;)")
    print(" для типа v14 игроку 2 выгоднее b1, так как  $8 > 4$ .)")
    print(" Поэтому стратегия Player2: v13->b2, v14->b1.")
    print("  $U1(a1) = 6p + 9(1-p) = 9 - 3p$ ")
    print(" При отклонении Player1 к a2:  $U1(a2) = 0 \cdot p + 2(1-p) = 2 - 2p$ ")
    print(" Условие a1 как лучшего ответа:  $9 - 3p \geq 2 - 2p \Rightarrow p \leq 7$ ")
    print(" Для всех  $p$  in  $[0,1]$  это выполнено.")

    print("\nЕсли Player1 выбирает a2:")
    print(" для типа v13 игроку 2 выгоднее b1, так как  $7 > 5$ ;)")
    print(" для типа v14 игроку 2 выгоднее b2, так как  $3 > 1$ .)")
    print(" Поэтому стратегия Player2: v13->b1, v14->b2.")
    print("  $U1(a2) = 11p + 10(1-p) = 10 + p$ ")
    print(" При отклонении Player1 к a1:  $U1(a1) = 4p + 7(1-p) = 7 - 3p$ ")
    print(" Условие a2 как лучшего ответа:  $10 + p \geq 7 - 3p \Rightarrow p \geq -3/4$ ")
    print(" Для всех  $p$  in  $[0,1]$  это выполнено.")

    print("\nСледовательно, при  $0 < p < 1$  статическая игра имеет два чистых BNE:")
    print(" 1) Player1->a1; Player2: v13->b2, v14->b1")
    print(" 2) Player1->a2; Player2: v13->b1, v14->b2")

def scan_static_methodical_p_grid() -> None:
    print("\n" + "-" * 92)
    print("Сканирование p для статической игры из вариантов 13 и 14")
    for p in [0.01, 0.25, 0.50, 0.75, 0.99]:

```

```

        game = build_static_methodical_variants_13_14_game(p)
        equilibria = game.bayes_nash_equilibria()
        print(f"p={p:.2f}: {len(equilibria)} BNE")
        for equilibrium in equilibria:
            print(
                f"    {game.describe_strategy_profile(equilibrium)};
E[u]={fmt_payoff(game.ex_ante_expected_payoff(equilibrium))}")

def scan_dynamic_methodical_p_grid() -> None:
    print("\n" + "-" * 92)
    print("Сканирование p для динамической игры из вариантов 13 и 14")
    for p in [0.00, 0.25, 0.50, 0.75, 1.00]:
        game = build_dynamic_methodical_variants_13_14_game(p)
        equilibria = game.pure_perfect_bayesian_equilibria()
        print(f"p={p:.2f}: {len(equilibria)} PBE")
        for action1, strategy2 in equilibria:
            print(f"    Player1->{action1}; E[u]={fmt_payoff(game.expected_payoff(action1,
strategy2))}")

def build_sheriff_game(q: float = 0.5) -> BayesianGame:
    if not (0.0 < q < 1.0):
        raise ValueError("Для статической модели используйте 0 < q < 1")

    players = ("Suspect", "Sheriff")
    types = (("criminal", "civilian"), ("sheriff",))
    actions = (("Shoot", "Not"), ("Shoot", "Not"))
    prior = {
        ("criminal", "sheriff"): float(q),
        ("civilian", "sheriff"): float(1 - q),
    }

    payoff: dict[tuple[TypeProfile, ActionProfile], Payoff] = {}

    criminal_matrix = {
        ("Shoot", "Shoot"): (0, 0),
        ("Shoot", "Not"): (2, -2),
        ("Not", "Shoot"): (-2, -1),
        ("Not", "Not"): (-1, 1),
    }
    civilian_matrix = {
        ("Shoot", "Shoot"): (-3, -1),
        ("Shoot", "Not"): (-1, -2),
        ("Not", "Shoot"): (-2, -1),
        ("Not", "Not"): (0, 0),
    }

    for action_profile, p in criminal_matrix.items():
        payoff[("criminal", "sheriff"), action_profile] = p
    for action_profile, p in civilian_matrix.items():
        payoff[("civilian", "sheriff"), action_profile] = p

    return BayesianGame(players, types, actions, prior, payoff)

def print_slide_threshold_example() -> None:
    print("\n" + "-" * 92)
    print("Контрольный пример со слайдов 13-14: порог по представлению  $p=P(t_1)$ ")
    print("Сравниваются два кандидата: (I, C) и (H/I, H/C).")
    print("U1(I, C) =  $2p + (-3)(1-p) = 5p - 3$ ")
    print("U1(H/I, H/C) =  $(-1)p + 0(1-p) = -p$ ")
    print("I выгоднее, если  $5p - 3 > -p$ , то есть  $p > 1/2$ .")

    for p in [0.25, 0.50, 0.75]:
        u_i = 5 * p - 3
        u_h = -p
        if abs(u_i - u_h) < 1e-9:
            conclusion = "игрок 1 безразличен; возможны оба кандидата"
        elif u_i > u_h:
            conclusion = "выбирается кандидат (I, C)"
        else:
            conclusion = "выбирается кандидат (H/I, H/C)"
        print(f"p={p:.2f}: U1(I,C)={fmt_number(u_i)}, U1(H/I,H/C)={fmt_number(u_h)} ->
{conclusion}")

def build_market_entry_game(p_normal: float = 0.7) -> DynamicTwoPlayerBayesianGame:
    if not (0.0 <= p_normal <= 1.0):

```

```

        raise ValueError("p_normal должна лежать в [0, 1]")

    payoff: dict[tuple[TypeName, Action, Action], Payoff] = {}
    for type_2 in ("Normal", "Tough"):
        for action2 in ("Fight", "Acquiesce"):
            payoff[(type_2, "Out", action2)] = (0, 2)

    payoff[("Normal", "In", "Fight")] = (-1, -1)
    payoff[("Normal", "In", "Acquiesce")] = (1, 1)

    payoff[("Tough", "In", "Fight")] = (-1, 1)
    payoff[("Tough", "In", "Acquiesce")] = (1, -1)

    return DynamicTwoPlayerBayesianGame(
        player1="Entrant",
        player2="Incumbent",
        player1_actions=("Out", "In"),
        player2_actions=("Fight", "Acquiesce"),
        player2_types=("Normal", "Tough"),
        prior={"Normal": float(p_normal), "Tough": float(1 - p_normal)},
        payoff=payoff,
    )

def solve_bayesian_task() -> None:
    print("\n" + "#" * 92)
    print("ОСНОВНОЙ РАСЧЕТ: варианты 13 и 14 из методички")
    print("#" * 92)

    static_game = build_static_methodical_variants_13_14_game(p_variant13=0.5)
    static_game.print_equilibrium_analysis(
        "Статическая байесовская игра: варианты 13 и 14, p=P(v13)=0.5"
    )
    print_static_threshold_for_methodical_game()
    scan_static_methodical_p_grid()

    dynamic_game = build_dynamic_methodical_variants_13_14_game(p_variant13=0.5)
    dynamic_game.print_backward_induction_solution(
        "Динамическая байесовская игра: варианты 13 и 14, p=P(v13)=0.5"
    )
    scan_dynamic_methodical_p_grid()

    print("\n" + "#" * 92)
    print("КОНТРОЛЬНЫЕ ЛЕКЦИОННЫЕ ПРИМЕРЫ")
    print("#" * 92)

    sheriff = build_sheriff_game(q=0.5)
    sheriff.print_equilibrium_analysis("Контрольный пример: дилемма шерифа, q=0.5")
    print_slide_threshold_example()

    market = build_market_entry_game(p_normal=0.7)
    market.print_backward_induction_solution("Контрольный динамический пример: выход на рынок, p(Normal)=0.7")

if __name__ == "__main__":
    solve_bayesian_task()

```